

Chapter 7 Categorical Data Analysis

Everything analysis covered so far has used a continuous variable as a target variable of interest. What if our target variable was categorical instead of continuous? Our analysis must change to adjust.

This Chapter aims to answer the following questions:

- How do you explore categorical variables?
 - Nominal vs. Ordinal
 - Tests of Association
 - Measures of Association
- How do you model a categorical target variable?
 - Logistic Regression
 - Interpreting Logistic Regression
 - Assessing Logistic Regression

7.1 Describing Categorical Data

We need to first explore our data before building any models to try and explain/predict our categorical target variable. With categorical variables, we can look at the distribution of the categories as well as see if this distribution has any association with other variables. For this analysis we are going to still use our Ames housing data. Imagine you worked for a real estate agency and got a bonus check if you sold a house above \$175,000 in value. Let's create this variable in our data:

```
library(dplyr)

train <- train %>%
  mutate(Bonus = ifelse(Sale_Price > 175000, 1, 0))
```

You are interested in what variables might be associated with obtaining a higher chance of getting a bonus (selling a house above \$175,000). An association exists between two categorical variables if the distribution of one variable changes when the value of the other categorical changes. If there is no association, the distribution of the first variable is the same regardless of the value of the other variable. For example, if we wanted to know if obtaining a bonus on selling a house in Ames, Iowa was associated with whether the house had central air we could look at the distribution of bonus eligible houses. If we observe that 42% of homes with central air are bonus eligible and 42% of homes without central air are bonus eligible, then it appears that central air has no bearing on whether the home is bonus eligible. However, if instead we observe that only 3% of homes without central air are bonus eligible, but 44% of home with central air are bonus eligible, then it appears that having central air might be related to a home being bonus eligible.

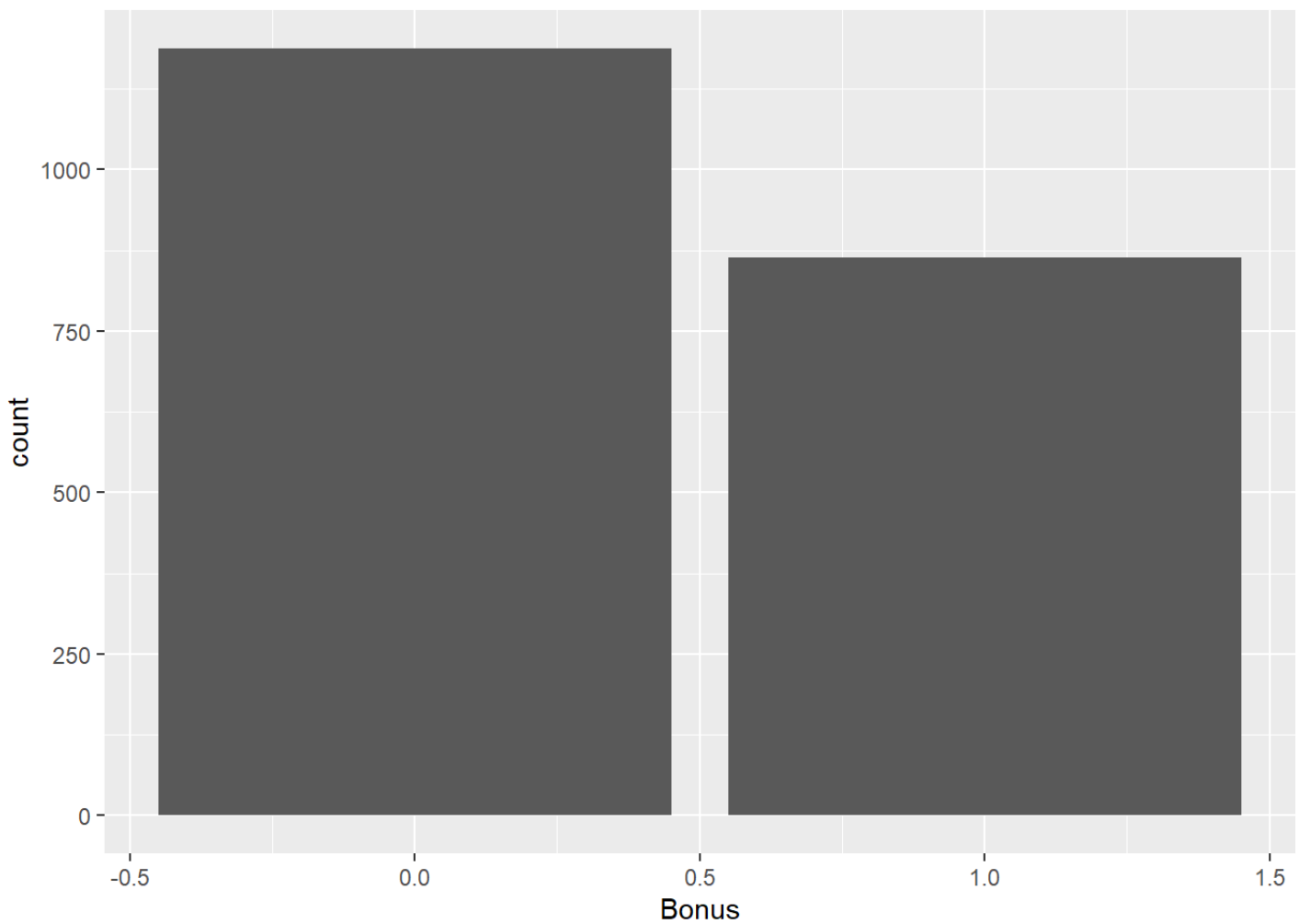
To understand the distribution of categorical variables we need to look at frequency tables. A frequency table shows the number of observations that occur in certain categories or intervals. A one way frequency table examines all the categories of one variable. These are easily visualized with bar charts.

Let's look at the distribution of both bonus eligibility and central air using the `table` function. The `ggplot` function with the `geom_bar` function allows us to view our data in a bar chart.

```
table(train$Bonus)
```

```
##  
##      0      1  
## 1187  864
```

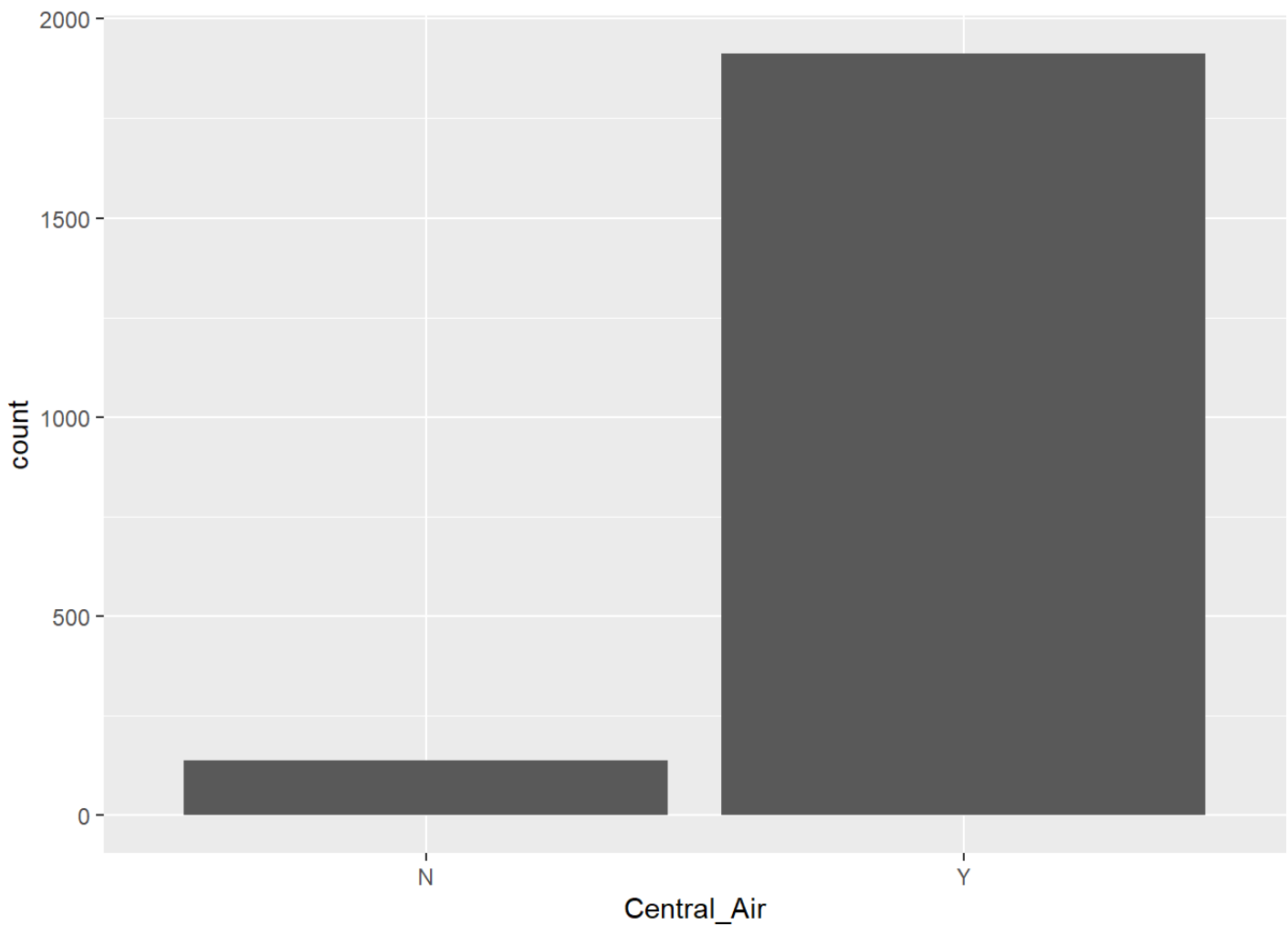
```
ggplot(data = train) +  
  geom_bar(mapping = aes(x = Bonus))
```



```
table(train$Central_Air)
```

```
##  
##      N      Y  
## 139 1912
```

```
ggplot(data = train) +  
  geom_bar(mapping = aes(x = Central_Air))
```



Frequency tables show single variables, but if we want to explore two variables together we look at **cross-tabulation** tables. A cross-tabulation table shows the number of observations for each combination of the row and column variables.

Let's again examine bonus eligibility, but this time across levels of central air. Again, we can use the `table` function. The `prop.table` function allows us to compare two variables in terms of proportions instead of frequencies.

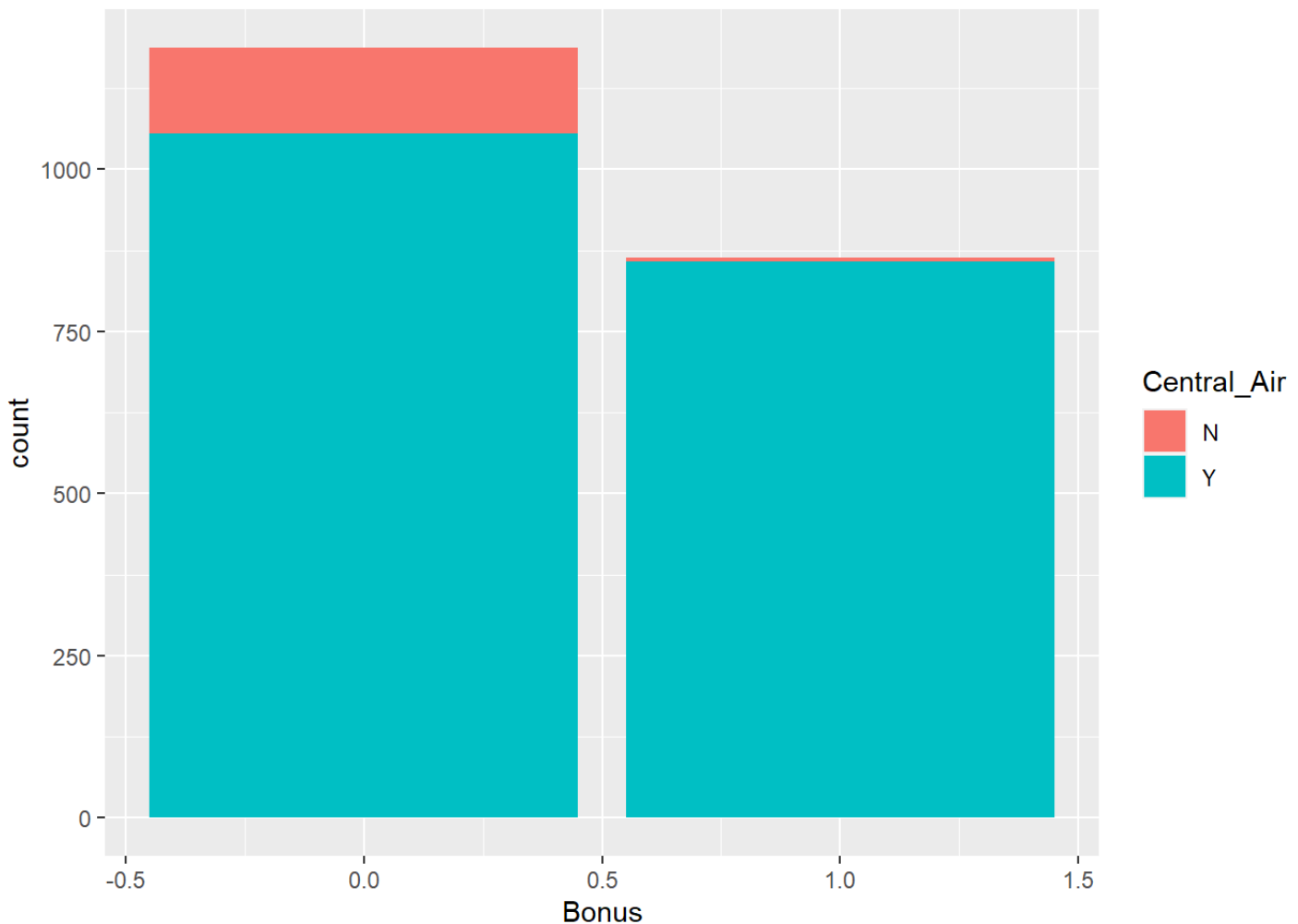
```
table(train$Central_Air, train$Bonus)
```

```
##
##           0      1
##    N  132     7
##    Y 1055   857
```

```
prop.table(table(train$Central_Air, train$Bonus))
```

```
##
##           0           1
##    N 0.064358849 0.003412969
##    Y 0.514383228 0.417844954
```

```
ggplot(data = train) +
  geom_bar(mapping = aes(x = Bonus, fill = Central_Air))
```



From the above output we can see that 147 homes have no central air with only 5 of them being bonus eligible. However, there are 1904 homes that have central air with 835 of them being bonus eligible. For an even more detailed breakdown we can use the `CrossTable` function.

```
library(gmodels)
```

```
CrossTable(train$Central_Air, train$Bonus)
```

```
##
```

```
##
```

```
##      Cell Contents
```

```
## |-----|
## |                N |
## | Chi-square contribution |
## |      N / Row Total |
## |      N / Col Total |
## |      N / Table Total |
## |-----|
##
##
## Total Observations in Table:  2051
##
##
##           | train$Bonus
## train$Central_Air |      0 |      1 | Row Total |
## -----|-----|-----|-----|
##           N |    132 |      7 |    139 |
##           |    33.040 |    45.392 |          |
##           |    0.950 |    0.050 |    0.068 |
##           |    0.111 |    0.008 |          |
##           |    0.064 |    0.003 |          |
## -----|-----|-----|-----|
##           Y |    1055 |     857 |    1912 |
##           |    2.402 |    3.300 |          |
##           |    0.552 |    0.448 |    0.932 |
##           |    0.889 |    0.992 |          |
##           |    0.514 |    0.418 |          |
## -----|-----|-----|-----|
##      Column Total |    1187 |     864 |    2051 |
##           |    0.579 |    0.421 |          |
## -----|-----|-----|-----|
##
```

##

The advantage of the `CrossTable` function is that we can easily get not only the frequencies, but the cell, row, and column proportions. For example, the third number in each cell gives us the row proportion. For homes without central air, 96.6% of them are not bonus eligible, while 3.4% of them are. For homes with central air, 56.1% of the homes are not bonus eligible, while 43.9% of them are. This would appear that the distribution of bonus eligible homes changes across levels of central air - a relationship between the two variables. This expected relationship needs to be tested statistically for verification.

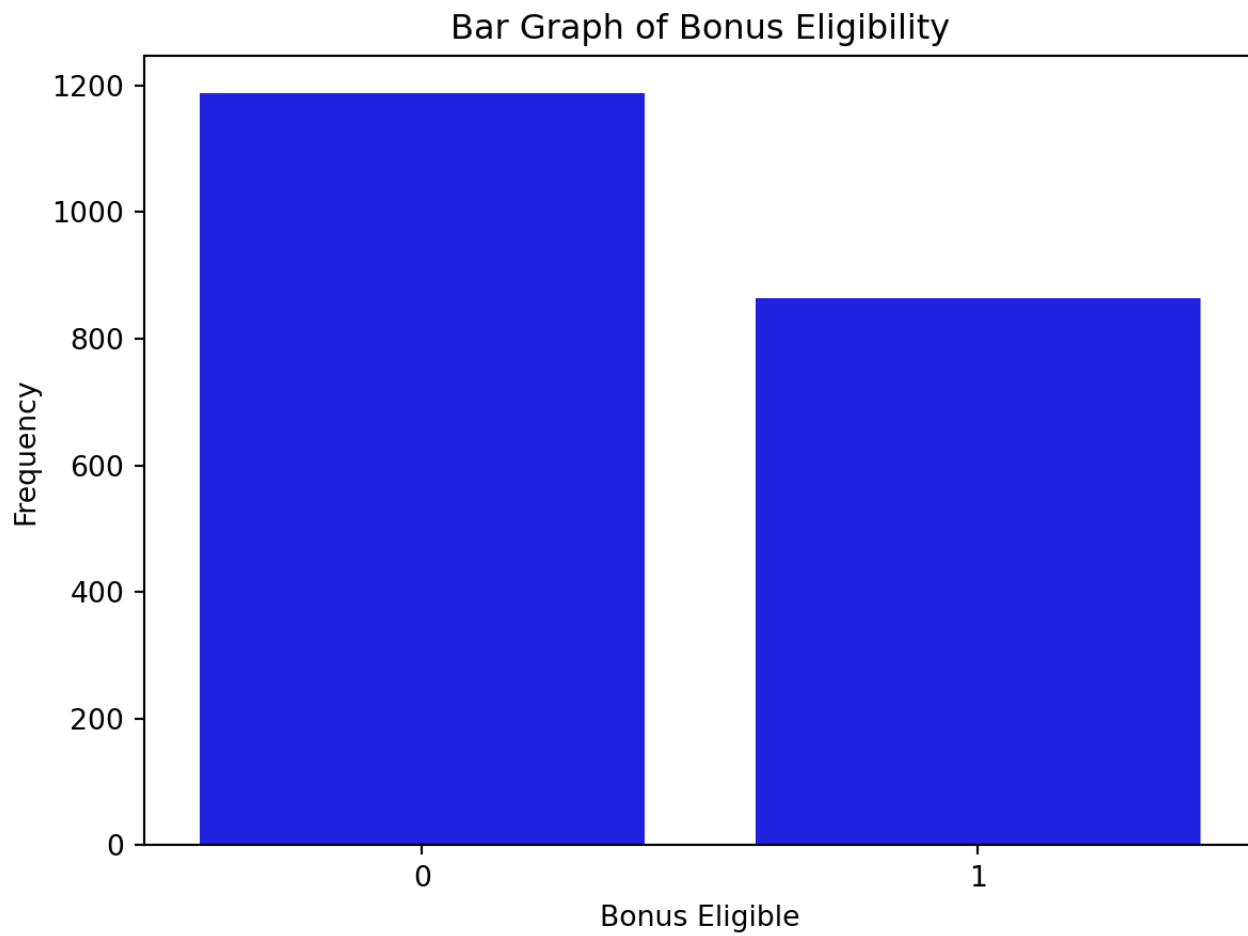
7.1.1 Python Code

```
train['Bonus'] = np.where(train['Sale_Price'] > 175000, 1, 0)
```



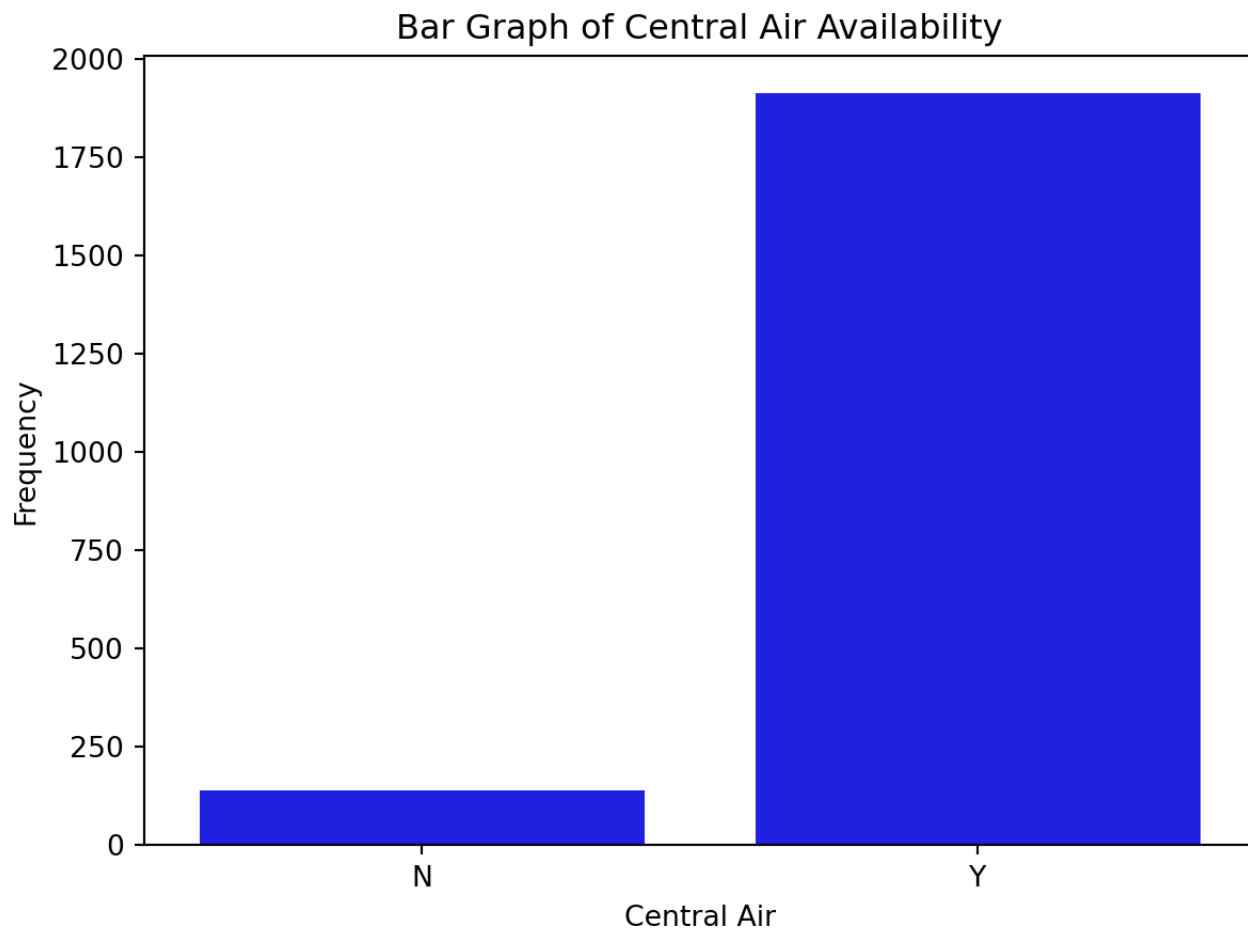
```
train['Bonus'].value_counts()

ax = sns.countplot(x = "Bonus", data = train, color = "blue")
ax.set(xlabel = 'Bonus Eligible',
       ylabel = 'Frequency',
       title = 'Bar Graph of Bonus Eligibility')
plt.show()
```

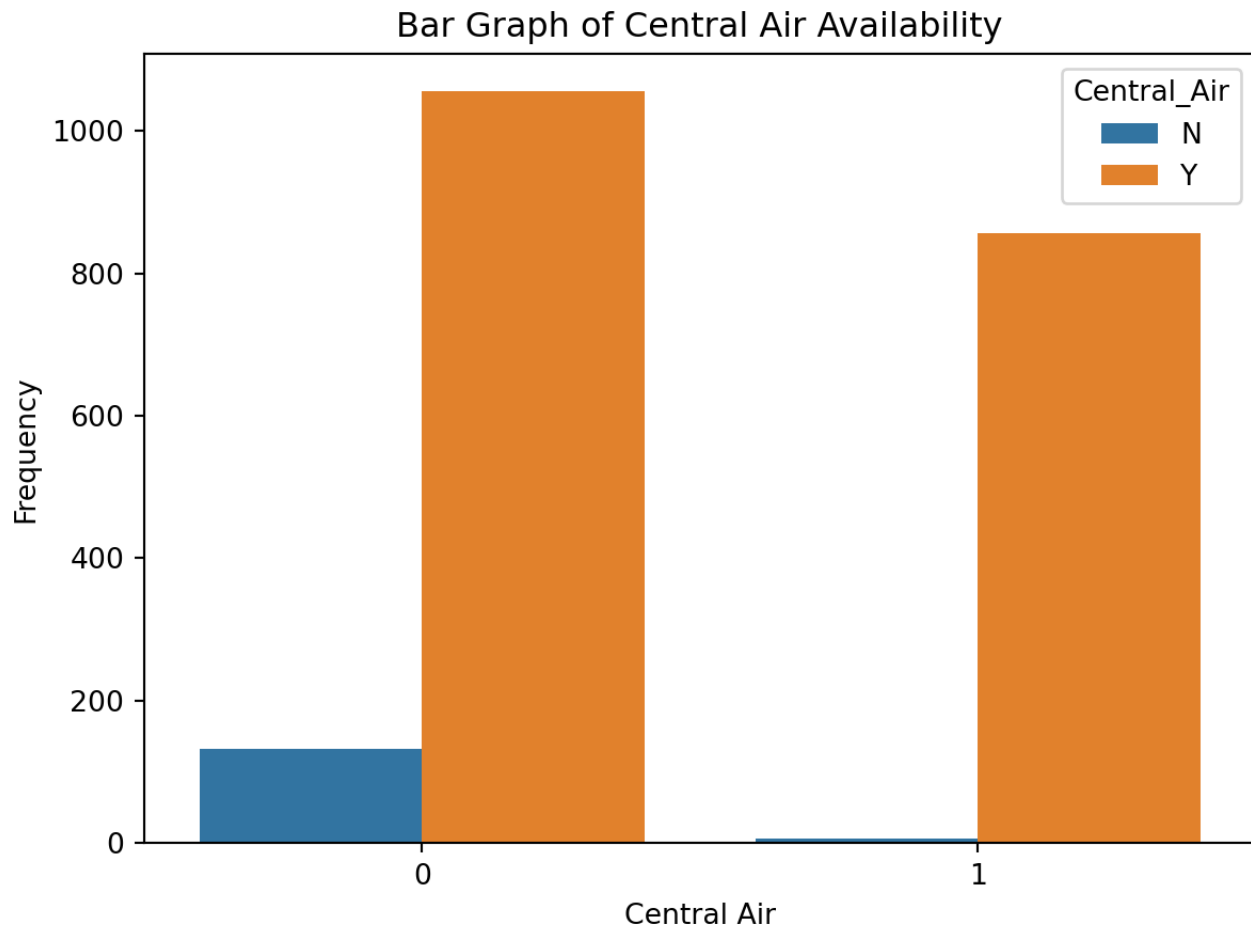



```
train['Central_Air'].value_counts()

plt.cla()
ax = sns.countplot(x = "Central_Air", data = train, color = "blue")
ax.set(xlabel = 'Central Air',
       ylabel = 'Frequency',
       title = 'Bar Graph of Central Air Availability')
plt.show()
```



```
plt.cla()
ax = sns.countplot(x = "Bonus", data = train, hue = "Central_Air")
ax.set(xlabel = 'Central Air',
       ylabel = 'Frequency',
       title = 'Bar Graph of Central Air Availability')
plt.show()
```



```
pd.crosstab(index = train['Central_Air'], columns = train['Bonus'])
```

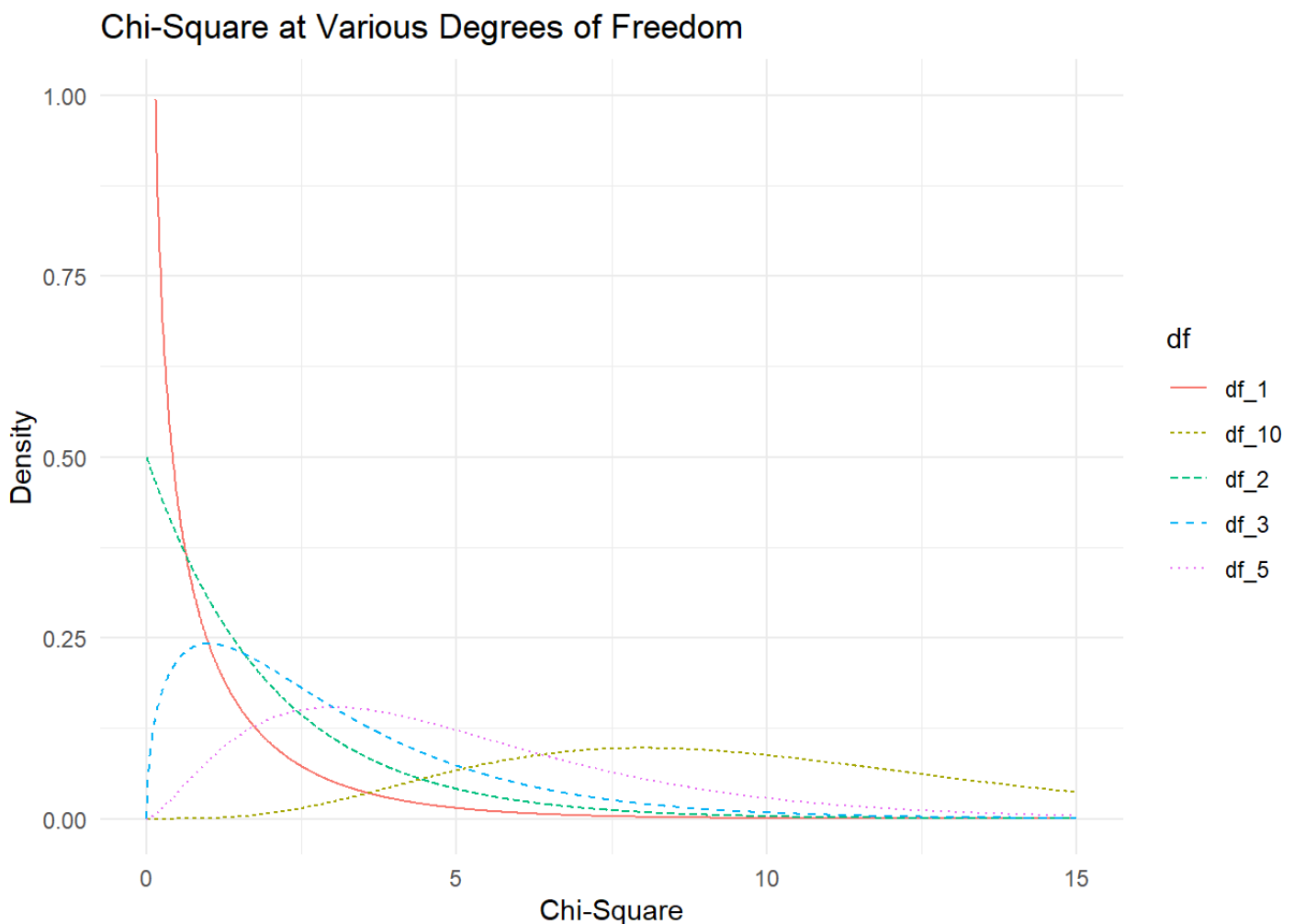
7.2 Tests of Association

Much like in Chapter 2 we have statistical tests to evaluate relationships between two categorical variables. The null hypothesis for these statistical tests is that the two variables have no association - the distribution of one variable does not change across levels of another variable. The alternative hypothesis is an association between the two variables - the distribution of one variable changes across levels of another variable.

These statistical tests follow a χ^2 -distribution. The χ^2 -distribution is a distribution that has the following characteristics:

- Bounded below by 0
- Right-skewed
- One set of degrees of freedom

A plot of a variety of χ^2 -distributions is shown here:



Two common χ^2 tests are the Pearson and Likelihood Ratio χ^2 tests. They compare the observed count of observations in each cell of a cross-tabulation table between two variables to their expected count **if** there was no relationship. The expected cell count applies the overall distribution of one variable across all the levels of the other variable. For example, overall 59% of all homes are not bonus eligible. **If** that were to apply to every level of central air, then the 140 homes without central air would be expected to have 86.73 ($59\% \times 147$) of them would be bonus eligible while 60.27 ($41\% \times 147$) of them would not be bonus eligible. We actually observe 142 and 5 homes for each of these categories respectively. The further the observed data is from the expected data, the more evidence we have that there is a relationship between the two variables.

The test statistic for the Pearson χ^2 test is the following:

$$\chi_P^2 = \sum_{i=1}^R \sum_{j=1}^C \frac{(Obs_{i,j} - Exp_{i,j})^2}{Exp_{i,j}}$$

From the equation above, the closer that the observed count of each cross-tabulation table cell to the expected count, the smaller the test statistic. As with all previous hypothesis tests, the smaller the test statistic, the larger the p-value, implying less evidence for the alternative hypothesis.

Let's examine the relationship between central air and bonus eligibility using the `chisq.test` function.

```
chisq.test(table(train$Central_Air, train$Bonus))
```

```
##
## Pearson's Chi-squared test with Yates' continuity
## correction
##
## data:  table(train$Central_Air, train$Bonus)
## X-squared = 82.509, df = 1, p-value < 2.2e-16
```

The above results shows an extremely small p-value that is below any reasonable significance level. This implies that we have statistical evidence for a relationship between having central air and bonus eligibility of homes. The p-value comes from a χ^2 -distribution with degrees of freedom that equal the product of the number of rows minus one and the number of columns minus one.

Another common test is the Likelihood Ratio test. The test statistic for this is the following:

$$\chi_L^2 = 2 \times \sum_{i=1}^R \sum_{j=1}^C Obs_{i,j} \times \log\left(\frac{Obs_{i,j}}{Exp_{i,j}}\right)$$

The p-value comes from a χ^2 -distribution with degrees of freedom that equal the product of the number of rows minus one and the number of columns minus one. Both of the above tests have a sample size requirement. The sample size requirement is 80% or more of the cells in the cross-tabulation table need **expected** count larger than 5.

For smaller sample sizes, this might be hard to meet. In those situations, we can use a more computationally expensive test called Fisher's exact test. This test calculates every possible permutation of the data being evaluated to calculate the p-value without any distributional assumptions. To perform this test we can use the `fisher.test` function.

```
fisher.test(table(train$Central_Air, train$Bonus))
```

```
##
## Fisher's Exact Test for Count Data
##
## data:  table(train$Central_Air, train$Bonus)
## p-value < 2.2e-16
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##   7.170439 39.018479
## sample estimates:
## odds ratio
##   15.30751
```

We see the same results as with the Pearson test because the assumptions were met for sample size.

Both the Pearson and Likelihood Ratio χ^2 tests can handle any type of categorical variable either ordinal, nominal, or both. However, ordinal variables provide us extra information since the order of the categories actually matters compared to nominal. We can test for even more with ordinal variables against other ordinal variables whether two ordinal variables have a **linear relationship** as compared to just a general one. An ordinal test for association is the Mantel-Haenszel χ^2 test. The test statistic for the Mantel-Haenszel χ^2 test is the following:

$$\chi_{MH}^2 = (n - 1)r^2$$

where r^2 is the Pearson correlation between the column and row variables. This test follows a χ^2 -distribution with only one degree of freedom.

Since both the central air and bonus eligibility variables are binary, they are ordinal. Since they are both ordinal, we should use the Mantel-Haenszel χ^2 test with the `CMHtest` function. In the main output table, the first row is the Mantel-Haenszel χ^2 test.

```
library(vcdExtra)

CMHtest(table(train$Central_Air, train$Bonus))$table[1,]

##           Chisq           Df           Prob
## 8.409247e+01 1.000000e+00 4.721656e-20
```

From here we can see another extremely small p-value as we saw in earlier, more general χ^2 tests.

7.2.1 Python Code

```
from scipy.stats import chi2_contingency

chi2_contingency(pd.crosstab(index = train['Central_Air'], columns = train['Bonu
```

```
from scipy.stats import fisher_exact
```

```
fisher_exact(pd.crosstab(index = train['Central_Air'], columns = train['Bonus']))
```

No real Mantel-Haenszel options in Python that work for anything more than a 2x2 table so I wouldn't trust them.

7.3 Measures of Association

Tests of association are best designed for just that, testing the existence of an association between two categorical variables. However, just like we saw in Chapter 1.1, hypothesis tests are impacted by sample size. When we have the same sample size, tests of association can rank significance of variables with p-values. However, when sample sizes are not the same between two tests, the tests of association are not best for comparing the strength of an association. In those scenarios, we have measures of strength of association that can be compared across any sample size.

Measures of association were not designed to test if an association exists, as that is what statistical testing is for. They are designed to measure the strength of association. There are dozens of these measures. Three of the most common are the following:

- Odds Ratios (only for comparing two binary variables)
- Cramer's V (able to compare nominal variables with any number of categories)
- Spearman's Correlation (able to compare ordinal variables with any number of categories)

An **odds ratio** indicates how much more likely, with respect to **odds**, a certain event occurs in one group relative to its occurrence in another group. The odds of an event occurring is *not* the same as the probability that an event occurs. The odds of an event occurring is the probability the event occurs divided by the probability that event does not occur.

$$Odds = \frac{p}{1 - p}$$

Let's again examine the cross-tabulation table between central air and bonus eligibility.

```
##
##
##      Cell Contents
## |-----|
## |                      N |
## | Chi-square contribution |
## |          N / Row Total |
## |          N / Col Total |
## |          N / Table Total |
## |-----|
##
##
## Total Observations in Table:  2051
##
##
```



```
##          | train$Bonus
## train$Central_Air |          0 |          1 | Row Total |
## -----|-----|-----|-----|
##          N |          132 |          7 |          139 |
##          |          33.040 |          45.392 |          |
##          |          0.950 |          0.050 |          0.068 |
##          |          0.111 |          0.008 |          |
##          |          0.064 |          0.003 |          |
## -----|-----|-----|-----|
##          Y |          1055 |          857 |          1912 |
##          |          2.402 |          3.300 |          |
##          |          0.552 |          0.448 |          0.932 |
##          |          0.889 |          0.992 |          |
##          |          0.514 |          0.418 |          |
## -----|-----|-----|-----|
##      Column Total |          1187 |          864 |          2051 |
##          |          0.579 |          0.421 |          |
## -----|-----|-----|-----|
##
##
```

Let's look at the row without central air. The probability that a home without central air is not bonus eligible is 96.6%. That implies that the odds of not being bonus eligible in homes without central air is 28.41 ($= 0.966/0.034$). For homes with central air, the odds of not being bonus eligible are 1.28 ($= 0.561/0.439$). The odds ratio between these two would be approximately 22.2 ($= 28.41/1.28$). In other words, homes without central air are 22.2 times more likely (in terms of odds) to not be bonus eligible as compared to homes with central air. This relationship is intuitive based on the numbers we have seen. Without going into details, it can also be shown that homes with central air are 22.2 times as likely (in terms of odds) to be bonus eligible.

We can use the `OddsRatio` function to get these same results.

```
library(DescTools)
```

```
OddsRatio(table(train$Central_Air, train$Bonus))
```

```
## [1] 15.31808
```

Cramer's V is another measure of strength of association. Cramer's V is calculated as follows:

$$V = \sqrt{\frac{\chi_P^2/n}{\min(Rows - 1, Columns - 1)}}$$

Cramer's V is bounded between 0 and 1 for every comparison other than two binary variables. For two binary variables being compared the bounds are -1 to 1. The idea is still the same for both. The further the value is from 0, the stronger the relationship. Unfortunately, unlike R^2 , Cramer's V has no interpretative value. It can only be used for comparison.

We use the `assocstats` function to get the Cramer's V value. This function also provides the Pearson and Likelihood Ratio χ^2 tests as well.

```
assocstats(table(train$Central_Air, train$Bonus))
```

```
##                X^2 df P(> X^2)
## Likelihood Ratio 106.675  1      0
## Pearson          84.133  1      0
##
## Phi-Coefficient   : 0.203
## Contingency Coeff.: 0.199
## Cramer's V        : 0.203
```

Lastly, we have Spearman's correlation. Much like the Mantel-Haenszel test of association was specifically designed for comparing two ordinal variables, Spearman correlation measures the strength of association between two ordinal variables. Spearman is not limited to only categorical data analysis as it was also seen back in Chapter 5 with detecting heteroskedasticity. Remember, Spearman correlation is a correlation on the ranks of the observations as compared to the actual values of the observations.

The `cor.test` function that gave us Pearson's correlation also provides Spearman's correlation.

```
cor.test(x = as.numeric(ordered(train$Central_Air)),
         y = as.numeric(ordered(train$Bonus)),
         method = "spearman")
```

```
##  
## Spearman's rank correlation rho  
##  
## data:  x and y  
## S = 1146718629, p-value < 2.2e-16  
## alternative hypothesis: true rho is not equal to 0  
## sample estimates:  
##      rho  
## 0.2025357
```

As previously mentioned, these are only a few of the dozens of different measures of association that exist. However, they are the most used ones.

7.3.1 Python Code

Odds Ratios are the statistic calculated from the Fisher's Exact test from the previous code:

```
from scipy.stats import fisher_exact  
  
fisher_exact(pd.crosstab(index = train['Central_Air'], columns = train['Bonus']))
```

```
from scipy.stats import fisher_exact
```

```
fisher_exact(pd.crosstab(index = train['Central_Air'], columns = train['Bonus']))
```

```
from scipy.stats.contingency import association  
  
association(pd.crosstab(index = train['Central_Air'], columns = train['Bonus']),
```

```
from scipy.stats import spearmanr  
  
spearmanr(train['Central_Air'], train['Bonus'])
```

7.4 Introduction to Logistic Regression

After exploring the categorical target variable, we can move on to modeling the categorical target variable. Logistic regression is a fundamental statistical analysis for data science and analytics. It part of a class of modeling techniques known as classification models since they are trying to predict categorical target variables. This target variable can be binary, ordinal, or even nominal in its structure. The primary focus will be binary logistic regression. It is the most common type of logistic regression, and sets up the foundation for both ordinal and nominal logistic regression.

Ordinary least squares regression is not the best approach to modeling categorical target variables. Mathematically, it can be shown that with a binary target variable coded as 0 and 1, an OLS linear regression model will produce the **linear probability model**.

7.4.1 Linear Probability Model

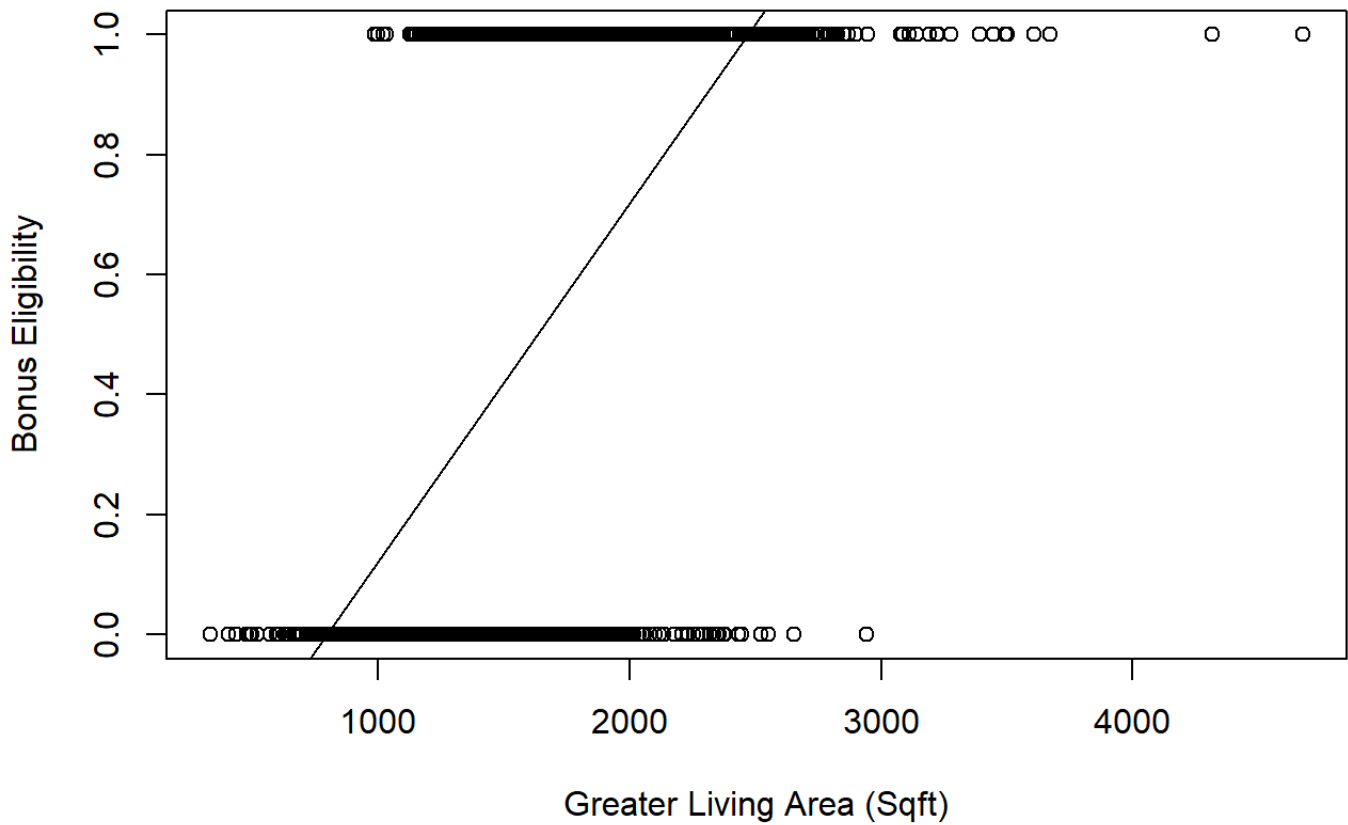
The linear probability model is not as widely used since probabilities do not tend to follow the properties of linearity in relation to their predictors. Also, the linear probability model possibly produces predictions outside of the bounds of 0 and 1 (where probabilities should be!). For completeness sake however, here is the linear probability model using the `lm` function to try and predict bonus eligibility.

```
lp.model <- lm(Bonus ~ Gr_Liv_Area, data = train)

with(train, plot(x = Gr_Liv_Area, y = Bonus,
                 main = 'OLS Regression?',
                 xlab = 'Greater Living Area (Sqft)',
                 ylab = 'Bonus Eligibility'))

abline(lp.model)
```

OLS Regression?



Even though it doesn't appear to really look like our data, let's fit this linear probability model anyway for completeness sake.

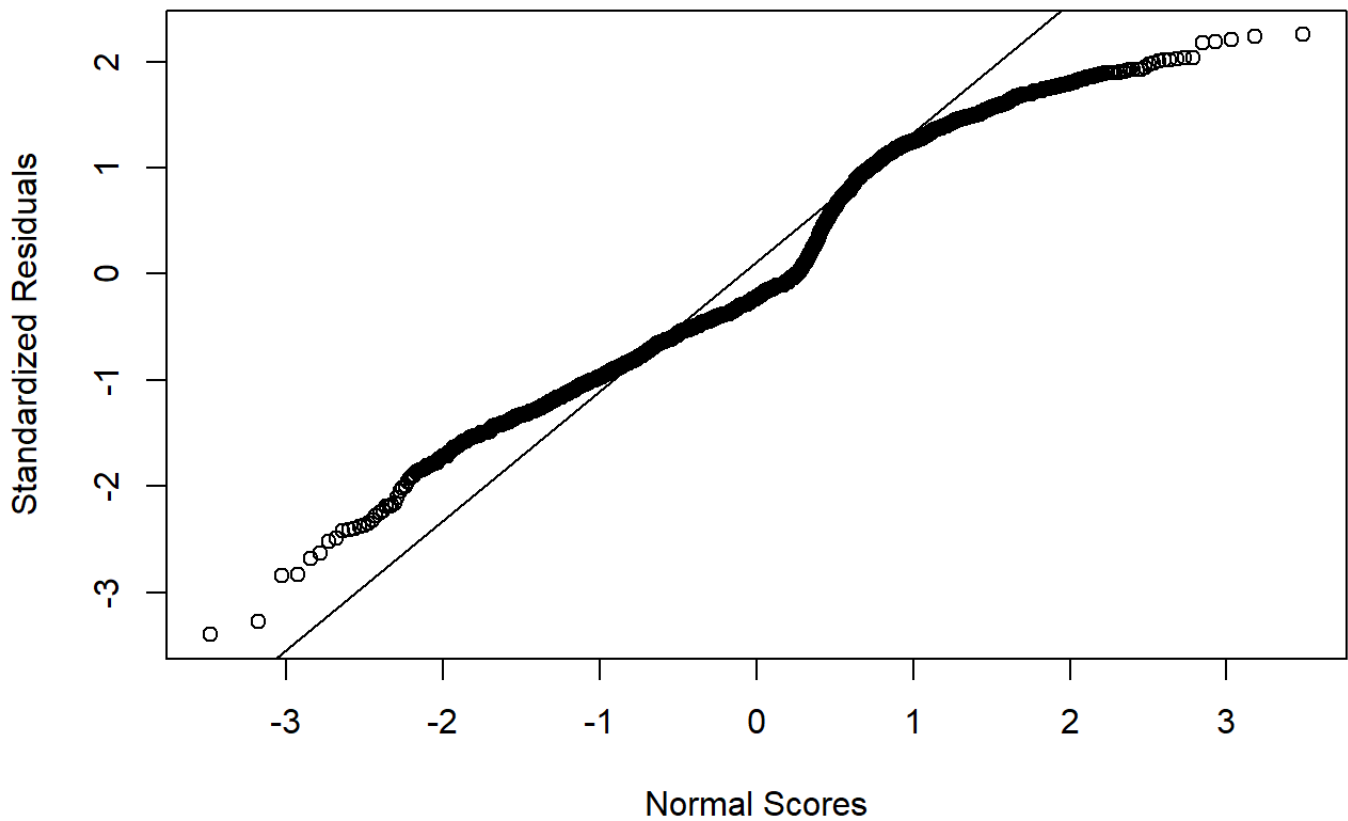
```
lp.model <- lm(Bonus ~ Gr_Liv_Area, data = train)
```

```
summary(lp.model)
```

```
##
## Call:
## lm(formula = Bonus ~ Gr_Liv_Area, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.32077 -0.27657 -0.08698  0.36907  0.88491
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -4.758e-01  2.737e-02  -17.39  <2e-16 ***
## Gr_Liv_Area  5.981e-04  1.731e-05   34.56  <2e-16 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3926 on 2049 degrees of freedom
## Multiple R-squared:  0.3682, Adjusted R-squared:  0.3679
## F-statistic: 1194 on 1 and 2049 DF, p-value: < 2.2e-16

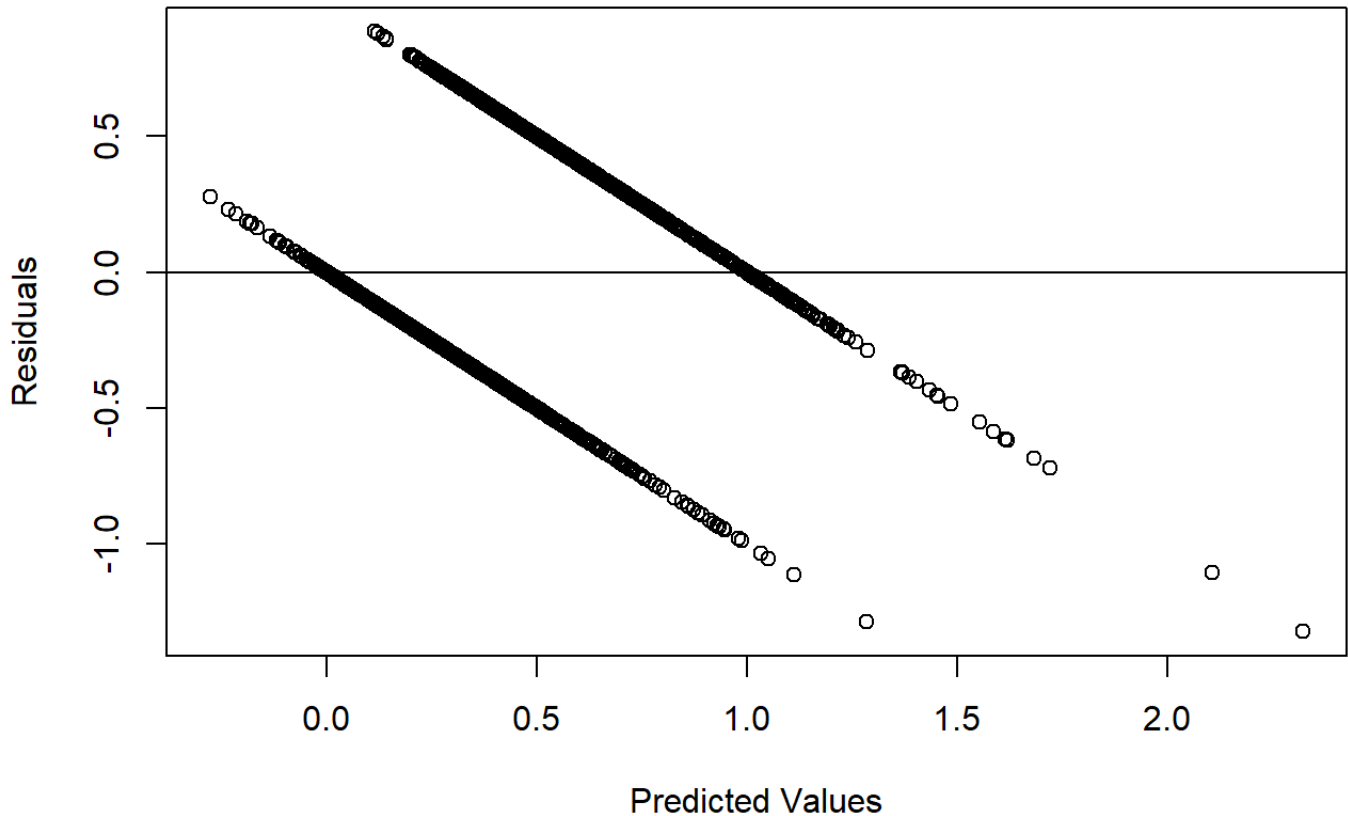
qqnorm(rstandard(lp.model),
       ylab = "Standardized Residuals",
       xlab = "Normal Scores",
       main = "QQ-Plot of Residuals")
qqline(rstandard(lp.model))
```

QQ-Plot of Residuals



```
plot(predict(lp.model), resid(lp.model),  
      ylab="Residuals", xlab="Predicted Values",  
      main="Residuals of Linear Probability Model")  
abline(0, 0)
```

Residuals of Linear Probability Model



As we can see from the charts above, the assumptions of ordinary least squares don't really hold in this situation. Therefore, we should be careful interpreting the results of the model. Maybe a better model won't have these problems?

7.4.2 Binary Logistic Regression

Due to the limitations of the linear probability model, people typically just use the binary logistic regression model. The logistic regression model does not have the limitations of the linear probability model. The outcome of the logistic regression model is the probability of getting a 1 in a binary variable. That probability is calculated as follows:

$$p_i = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_{1,i} + \dots + \beta_k x_{k,i})}}$$

This function has the desired properties for predicting probabilities. The predicted probability from the above equation will always be between 0 and 1. The parameter estimates do not enter the function linearly (this is a non-linear regression model), and the rate of change of the probability varies as the predictor variables vary as seen in Figure 7.1.

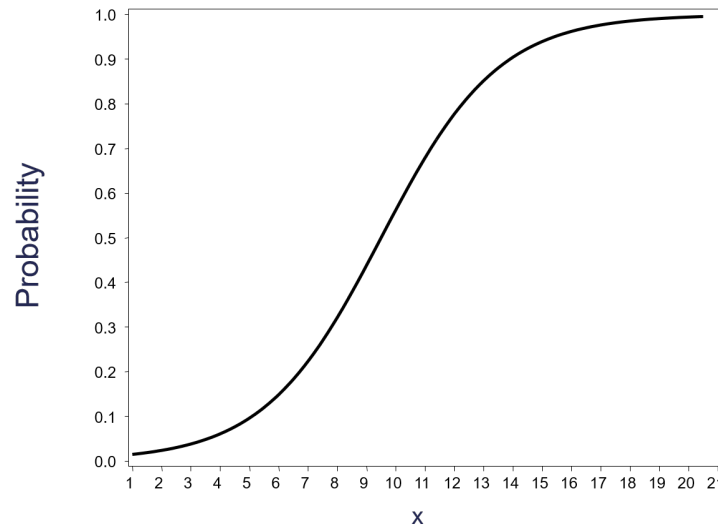


Figure 7.1: Example of a Logistic Curve

To create a linear model, a **link function** is applied to the probabilities. The specific link function for logistic regression is called the **logit** function.

$$\text{logit}(p_i) = \log\left(\frac{p_i}{1 - p_i}\right) = \beta_0 + \beta_1 x_{1,i} + \cdots + \beta_k x_{k,i}$$

The relationship between the predictor variables and the logits are linear in nature as the logits themselves are unbounded. This structure looks much more like our linear regression model structure. However, logistic regression does not use OLS to estimate the coefficients in our model. OLS requires residuals which the logistic regression model does not provide. The target variable is binary in nature, but the predictions are probabilities. Therefore, we cannot calculate a traditional residual. Instead, logistic regression uses maximum likelihood estimation. This is not covered here.

There are two main assumptions for logistic regression:

1. Independence of observations
2. Linearity of the logit

The first assumption of independence is the same as we had for linear regression. The second assumption implies that the logistic function transformation (the logit) actually makes a linear relationship with our predictor variables. This assumption can be tested, but will not be covered in this brief introduction to logistic regression.

Let's build a logistic regression model. We will use the `glm` function to do this. The `glm` function has a similar structure to the `lm` function. The main difference is the `family = binomial(link = "logit")` option to specify that we are using a logistic regression model. Again, there are many different link functions, but only the logistic link function (the logit) is being used here.

```
ames_logit <- glm(Bonus ~ Gr_Liv_Area,  
                  data = train, family = binomial(link = "logit"))  
summary(ames_logit)
```

```
##
## Call:
## glm(formula = Bonus ~ Gr_Liv_Area, family = binomial(link = "logit"),
##      data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -6.7458627  0.2976119  -22.67  <2e-16 ***
## Gr_Liv_Area  0.0042721  0.0001943   21.98  <2e-16 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 2792.2  on 2050  degrees of freedom
## Residual deviance: 1822.8  on 2049  degrees of freedom
## AIC: 1826.8
##
## Number of Fisher Scoring iterations: 5
```

Let's examine the above output. Scanning down the output, you can see the actual logistic regression equation for the variable `Gr_Liv_Area`. Here we can see that it appears to be a significant variable at predicting bonus eligibility. However, the coefficient reported does not have the same usable interpretation as in linear regression. An increase of one unit of greater living area square footage is linearly related to the logit **not** the probability of bonus eligibility. We can transform this coefficient to make it more interpretable. A single unit increase in greater living area square footage **does** have a $100 \times (e^{\hat{\beta}} - 1)\%$ increase in the average **odds** of bonus eligibility. We can use a combination of the `exp` and `coef` functions to obtain this number.


```
100*(exp(cbind(coef(ames_logit), confint(ames_logit)))-1)
```

```
## Waiting for profiling to be done...
```

```
##              2.5 %      97.5 %
## (Intercept) -99.8824266 -99.9353803 -99.7923687
## Gr_Liv_Area   0.4281269   0.3908381   0.4673827
```

In other words, every additional square foot in greater living area in the home leads to an average increase in odds of 0.385% to be bonus eligible.

7.4.3 Adding Categorical Variables

Similar to linear regression as we learned in Chapter 3, logistic regression can have both continuous and categorical predictors for our categorical target variable. Let's add both central air as well as number of fireplaces to our logistic regression model.

```
ames_logit2 <- glm(Bonus ~ Gr_Liv_Area + Central_Air + factor(Fireplaces),
                  data = train, family = binomial(link = "logit"))
summary(ames_logit2)
```

```
##
```

```
## Call:
```

```
## glm(formula = Bonus ~ Gr_Liv_Area + Central_Air + factor(Fireplaces),
##      family = binomial(link = "logit"), data = train)
```

```
##
```

```
## Coefficients:
```

```
##              Estimate Std. Error z value Pr(>|z|)
```

```
## (Intercept)          -1.000e+01  6.341e-01 -15.771  < 2e-16
## Gr_Liv_Area           4.106e-03  2.141e-04  19.179  < 2e-16
## Central_AirY          3.026e+00  4.971e-01   6.088  1.15e-09
## factor(Fireplaces)1   1.092e+00  1.292e-01   8.451  < 2e-16
## factor(Fireplaces)2   6.016e-01  2.419e-01   2.487   0.0129
## factor(Fireplaces)3  -4.183e-01  9.345e-01  -0.448   0.6544
## factor(Fireplaces)4   8.676e+00  3.247e+02   0.027   0.9787
##
## (Intercept)          ***
## Gr_Liv_Area           ***
## Central_AirY          ***
## factor(Fireplaces)1 ***
## factor(Fireplaces)2 *
## factor(Fireplaces)3
## factor(Fireplaces)4
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 2792.2  on 2050  degrees of freedom
## Residual deviance: 1662.8  on 2044  degrees of freedom
## AIC: 1676.8
##
## Number of Fisher Scoring iterations: 11
```

Just like with linear regression, categorical predictor variables are a comparison between two categories. Again, the coefficients from the logistic regression model need to be transformed to be interpreted.

```
100*(exp(cbind(coef(ames_logit2), confint(ames_logit2)))-1)
```

```
##                                2.5 %           97.5 %
## (Intercept)                -9.999547e+01  -99.9987897  -99.9852995
## Gr_Liv_Area                  4.114595e-01   0.3703791   0.4546903
## Central_AirY                 1.961924e+03  736.7096319 5882.6956323
## factor(Fireplaces)1          1.978790e+02  131.4232518 284.0534117
## factor(Fireplaces)2           8.250396e+01  13.6156598 193.6738150
## factor(Fireplaces)3          -3.418652e+01  -89.1376591 419.4010973
## factor(Fireplaces)4           5.860965e+05 -100.0000000      NA
```

Let's use the first fireplace variable as an example. A home with one fireplace has, on average, 167.04% higher odds of being bonus eligible as compared to a home with zero fireplaces.

7.4.4 Model Assessment

There are dozens of different ways to evaluate a logistic regression model. We will cover one popular way here - concordance. Counting the number of **concordant**, **discordant**, and **tied** pairs is a way to assess how well the model fits the data.

To find concordant, discordant, and tied pairs, we must compare all of the 0's in the target variable to all of the 1's. For our example, we will compare every pair of homes where one home is bonus eligible and one is not (every 0 and 1 pair). A **concordant** pair is a 0 and 1 pair where the bonus eligible home (the 1 in our model) has a higher predicted probability than the non-bonus eligible home (the 0 in our model) - our model successfully ordered these two observations by probability. It does not matter what the actual predicted probability values are as long as the bonus eligible home has a higher predicted probability than the non-bonus eligible home. A **discordant** pair is a 0 and 1 pair where the bonus eligible home (the 1 in our model) has a lower predicted probability than the non-bonus eligible home (the 0 in our

model) - our model unsuccessfully ordered the homes. It does not matter what the actual predicted probability values are as long as the bonus eligible home has a lower predicted probability than the non-bonus eligible home. A **tied** pair is a 0 and 1 pair where the bonus eligible home has the same predicted probability as the non-bonus eligible home - the model is confused and sees these two different things as the same. In general, you want a high percentage of concordant pairs and low percentages of discordant and tied pairs.

We can use the `Concordance` function to obtain these values on our predictions from the `predict` function.

```
library(survival)

survival::concordance(ames_logit)

## Call:
## concordance.lm(object = ames_logit)
##
## n= 2051
## Concordance= 0.8765 se= 0.007326
## concordant discordant      tied.x      tied.y      tied.xy
##      898616      126370        582    1074637        2070
```

From the above output we have a concordance of 86.3% for our model. There is no good or bad value as this can only be compared with another model to see which is better. Let's compare this to our model with the categorical variables.

```
survival::concordance(ames_logit2)
```

```
## Call:
## concordance.lm(object = ames_logit2)
##
## n= 2051
## Concordance= 0.8953 se= 0.006677
## concordant discordant      tied.x      tied.y      tied.xy
##      918066      107238        264      1075521        1186
```

We can see that the model with categorical predictors added to it has a higher concordance at 88.4%. That implies that our model is correctly able to rank our observations 88.4% of the time. This is **NOT** the same thing as saying our model is 88.4% accurate. Accuracy (which is not covered here) deals with a prediction being correct or incorrect. Concordance is only measuring how often we are able to predict 1's with higher probability than 0's - again, correctly ranking the observations.

7.4.5 Variable Selection and Regularized Regression

As with linear regression in Chapters ?? and ??, logistic regression uses the same approaches to doing variable selection. In fact, the same function are used as well. Let's use the `step` function to apply a forward and backward selection to the logistic regression model.

```
train_sel_log <- train %>%
  dplyr::select(Bonus,
    Lot_Area,
    Street,
    Bldg_Type,
    House_Style,
    Overall_Qual,
    Roof_Style,
    Central_Air,
    First_Flr_SF,
    Second_Flr_SF,
    Full_Bath,
    Half_Bath,
    Fireplaces,
    Garage_Area,
    Gr_Liv_Area,
    TotRms_AbvGrd) %>%
  replace(is.na(.), 0)

full.model <- glm(Bonus ~ . , data = train_sel_log)

empty.model <- glm(Bonus ~ 1, data = train_sel_log)

for.model <- step(empty.model,
  scope = list(lower = formula(empty.model),
    upper = formula(full.model)),
  direction = "forward", k = log(dim(train_sel_log)[1]))

## Start: AIC=2935.31
```

```
## Bonus ~ 1
##
##           Df Deviance    AIC
## + Overall_Qual    9   232.38 1432.3
## + Gr_Liv_Area     1   315.90 2001.0
## + Full_Bath       1   322.17 2041.3
## + Garage_Area     1   362.92 2285.6
## + First_Flr_SF    1   384.98 2406.6
## + TotRms_AbvGrd   1   414.37 2557.5
## + Fireplaces      1   415.58 2563.5
## + Half_Bath       1   458.95 2767.1
## + House_Style     7   449.42 2769.8
## + Second_Flr_SF   1   459.79 2770.8
## + Lot_Area        1   474.11 2833.8
## + Central_Air     1   479.52 2857.0
## + Bldg_Type       4   483.24 2895.8
## <none>           500.03 2935.3
## + Street          1   499.16 2939.4
## + Roof_Style      5   494.32 2949.9
##
## Step:  AIC=1432.29
## Bonus ~ Overall_Qual
##
##           Df Deviance    AIC
## + Gr_Liv_Area     1   201.93 1151.8
## + Full_Bath       1   203.53 1168.0
## + First_Flr_SF    1   214.05 1271.3
## + Lot_Area        1   216.67 1296.3
## + TotRms_AbvGrd   1   219.78 1325.5
## + Fireplaces      1   220.87 1335.7
## + Garage_Area     1   221.72 1343.5
```

```

## + Bldg_Type      4    225.06 1397.1
## + Second_Flr_SF  1    228.07 1401.5
## + Half_Bath      1    229.45 1413.9
## + Central_Air    1    231.20 1429.5
## <none>           232.38 1432.3
## + Street         1    232.38 1439.9
## + House_Style    7    227.39 1441.1
## + Roof_Style     5    231.04 1458.5
##
## Step:  AIC=1151.82
## Bonus ~ Overall_Qual + Gr_Liv_Area
##
##           Df Deviance    AIC
## + Full_Bath    1    192.70 1063.5
## + Lot_Area     1    195.87 1097.0
## + First_Flr_SF 1    195.95 1097.8
## + Second_Flr_SF 1    196.64 1105.0
## + House_Style  7    192.48 1106.9
## + Garage_Area  1    197.51 1114.1
## + Fireplaces   1    198.62 1125.6
## + Bldg_Type    4    198.14 1143.5
## + TotRms_AbvGrd 1    200.64 1146.3
## + Central_Air  1    200.72 1147.2
## <none>         201.93 1151.8
## + Half_Bath    1    201.75 1157.6
## + Street       1    201.91 1159.2
## + Roof_Style   5    200.63 1176.7
##
## Step:  AIC=1063.48
## Bonus ~ Overall_Qual + Gr_Liv_Area + Full_Bath
##

```



```

##                Df Deviance    AIC
## + Lot_Area      1   186.26 1001.4
## + First_Flr_SF  1   187.37 1013.6
## + Fireplaces    1   187.77 1018.0
## + Second_Flr_SF 1   187.92 1019.6
## + Bldg_Type     4   186.51 1027.1
## + House_Style   7   185.13 1034.7
## + Garage_Area   1   189.34 1035.0
## + TotRms_AbvGrd 1   190.75 1050.3
## + Central_Air   1   191.66 1060.0
## <none>          192.70 1063.5
## + Half_Bath     1   192.65 1070.6
## + Street        1   192.67 1070.8
## + Roof_Style    5   191.44 1088.1
##
## Step:  AIC=1001.43
## Bonus ~ Overall_Qual + Gr_Liv_Area + Full_Bath + Lot_Area
##
##                Df Deviance    AIC
## + Fireplaces    1   182.91  971.79
## + First_Flr_SF  1   182.96  972.34
## + Second_Flr_SF 1   183.35  976.69
## + Bldg_Type     4   181.35  977.07
## + Garage_Area   1   183.95  983.40
## + House_Style   7   180.20  986.97
## + TotRms_AbvGrd 1   184.44  988.88
## + Central_Air   1   185.43  999.86
## <none>          186.26 1001.43
## + Half_Bath     1   186.02 1006.36
## + Street        1   186.12 1007.46
## + Roof_Style    5   185.31 1029.05

```

##

Step: AIC=971.79

Bonus ~ Overall_Qual + Gr_Liv_Area + Full_Bath + Lot_Area + Fireplaces

##

##		Df	Deviance	AIC
##	+ Garage_Area	1	180.58	953.15
##	+ First_Flr_SF	1	180.59	953.22
##	+ Bldg_Type	4	178.81	955.84
##	+ Second_Flr_SF	1	180.89	956.60
##	+ TotRms_AbvGrd	1	181.41	962.48
##	+ House_Style	7	177.49	963.48
##	<none>		182.91	971.79
##	+ Central_Air	1	182.34	972.97
##	+ Half_Bath	1	182.61	976.04
##	+ Street	1	182.80	978.24
##	+ Roof_Style	5	182.01	999.85

##

Step: AIC=953.15

Bonus ~ Overall_Qual + Gr_Liv_Area + Full_Bath + Lot_Area + Fireplaces +

Garage_Area

##

##		Df	Deviance	AIC
##	+ Bldg_Type	4	176.89	941.30
##	+ First_Flr_SF	1	178.97	942.38
##	+ Second_Flr_SF	1	179.19	944.87
##	+ TotRms_AbvGrd	1	179.24	945.49
##	<none>		180.58	953.15
##	+ House_Style	7	176.22	956.41
##	+ Half_Bath	1	180.26	957.08
##	+ Central_Air	1	180.33	957.94
##	+ Street	1	180.44	959.21

```
## + Roof_Style      5    179.47 978.59
```

```
##
```

```
## Step:  AIC=941.3
```

```
## Bonus ~ Overall_Qual + Gr_Liv_Area + Full_Bath + Lot_Area + Fireplaces +
```

```
##      Garage_Area + Bldg_Type
```

```
##
```

```
##           Df Deviance    AIC
```

```
## + First_Flr_SF   1    175.26 929.97
```

```
## + Second_Flr_SF  1    175.51 932.90
```

```
## + TotRms_AbvGrd  1    175.78 936.06
```

```
## <none>           176.89 941.30
```

```
## + House_Style    7    172.53 943.47
```

```
## + Half_Bath      1    176.46 943.88
```

```
## + Central_Air    1    176.82 948.09
```

```
## + Street         1    176.84 948.38
```

```
## + Roof_Style     5    175.65 964.96
```

```
##
```

```
## Step:  AIC=929.97
```

```
## Bonus ~ Overall_Qual + Gr_Liv_Area + Full_Bath + Lot_Area + Fireplaces +
```

```
##      Garage_Area + Bldg_Type + First_Flr_SF
```

```
##
```

```
##           Df Deviance    AIC
```

```
## + Half_Bath      1    173.16 912.81
```

```
## + TotRms_AbvGrd  1    174.48 928.40
```

```
## <none>           175.26 929.97
```

```
## + Second_Flr_SF  1    175.00 934.56
```

```
## + Central_Air    1    175.22 937.05
```

```
## + Street         1    175.24 937.29
```

```
## + House_Style    7    172.23 947.58
```

```
## + Roof_Style     5    173.59 948.48
```

```
##
```

```
## Step:  AIC=912.81
## Bonus ~ Overall_Qual + Gr_Liv_Area + Full_Bath + Lot_Area + Fireplaces +
##      Garage_Area + Bldg_Type + First_Flr_SF + Half_Bath
##
##           Df Deviance    AIC
## + TotRms_AbvGrd  1   172.38 911.27
## <none>           173.16 912.81
## + Second_Flr_SF  1   173.05 919.20
## + Street         1   173.14 920.27
## + Central_Air    1   173.16 920.41
## + Roof_Style     5   171.28 928.53
## + House_Style    7   170.89 939.20
##
## Step:  AIC=911.27
## Bonus ~ Overall_Qual + Gr_Liv_Area + Full_Bath + Lot_Area + Fireplaces +
##      Garage_Area + Bldg_Type + First_Flr_SF + Half_Bath + TotRms_AbvGrd
##
##           Df Deviance    AIC
## <none>           172.38 911.27
## + Second_Flr_SF  1   172.28 917.58
## + Street         1   172.37 918.68
## + Central_Air    1   172.38 918.86
## + Roof_Style     5   170.66 928.75
## + House_Style    7   170.15 937.89

back.model <- step(full.model,
                    scope = list(lower = formula(empty.model),
                                  upper = formula(full.model)),
                    direction = "backward", k = log(dim(train_sel_log)[1]))
```

```
## Start:  AIC=974.27
## Bonus ~ Lot_Area + Street + Bldg_Type + House_Style + Overall_Qual +
##      Roof_Style + Central_Air + First_Flr_SF + Second_Flr_SF +
##      Full_Bath + Half_Bath + Fireplaces + Garage_Area + Gr_Liv_Area +
##      TotRms_AbvGrd
##
##              Df Deviance      AIC
## - House_Style    7   170.54  950.18
## - Roof_Style     5   170.02  959.15
## - Central_Air    1   168.12  966.66
## - Gr_Liv_Area    1   168.12  966.69
## - Street         1   168.14  966.86
## - Second_Flr_SF  1   168.21  967.76
## - First_Flr_SF   1   168.41  970.18
## - TotRms_AbvGrd  1   168.72  973.92
## <none>           168.12  974.27
## - Garage_Area    1   168.91  976.23
## - Half_Bath      1   169.43  982.57
## - Fireplaces     1   169.77  986.72
## - Bldg_Type      4   172.05  991.13
## - Lot_Area       1   170.32  993.32
## - Full_Bath      1   179.45 1100.39
## - Overall_Qual   9   213.18 1392.69
##
## Step:  AIC=950.18
## Bonus ~ Lot_Area + Street + Bldg_Type + Overall_Qual + Roof_Style +
##      Central_Air + First_Flr_SF + Second_Flr_SF + Full_Bath +
##      Half_Bath + Fireplaces + Garage_Area + Gr_Liv_Area + TotRms_AbvGrd
##
##              Df Deviance      AIC
## - Roof_Style     5   172.25  932.57
```

```
## - Central_Air      1    170.54  942.56
## - Street           1    170.55  942.68
## - Gr_Liv_Area      1    170.56  942.80
## - Second_Flr_SF    1    170.65  943.87
## - First_Flr_SF     1    171.01  948.22
## - TotRms_AbvGrd    1    171.16  950.06
## <none>              170.54  950.18
## - Garage_Area      1    171.55  954.72
## - Fireplaces       1    171.96  959.55
## - Bldg_Type         4    174.37  965.20
## - Half_Bath         1    172.62  967.49
## - Lot_Area          1    172.66  967.95
## - Full_Bath         1    183.19 1089.31
## - Overall_Qual      9    217.76 1382.84
##
```

```
## Step:  AIC=932.57
```

```
## Bonus ~ Lot_Area + Street + Bldg_Type + Overall_Qual + Central_Air +
##      First_Flr_SF + Second_Flr_SF + Full_Bath + Half_Bath + Fireplaces +
##      Garage_Area + Gr_Liv_Area + TotRms_AbvGrd
##
```

```
##           Df Deviance    AIC
## - Central_Air      1    172.26  924.99
## - Gr_Liv_Area      1    172.27  925.12
## - Street           1    172.27  925.16
## - Second_Flr_SF    1    172.37  926.28
## - First_Flr_SF     1    172.68  929.96
## <none>              172.25  932.57
## - TotRms_AbvGrd    1    173.04  934.25
## - Garage_Area      1    173.16  935.65
## - Fireplaces       1    173.72  942.31
## - Bldg_Type         4    175.89  944.95
```

```
## - Half_Bath      1    174.13  947.17
## - Lot_Area       1    174.55  952.09
## - Full_Bath      1    185.08 1072.29
## - Overall_Qual   9    219.97 1365.42
##
## Step:  AIC=924.99
## Bonus ~ Lot_Area + Street + Bldg_Type + Overall_Qual + First_Flr_SF +
##      Second_Flr_SF + Full_Bath + Half_Bath + Fireplaces + Garage_Area +
##      Gr_Liv_Area + TotRms_AbvGrd
##
##              Df Deviance      AIC
## - Gr_Liv_Area   1    172.27  917.53
## - Street        1    172.28  917.58
## - Second_Flr_SF  1    172.37  918.68
## - First_Flr_SF  1    172.68  922.36
## <none>          172.26  924.99
## - TotRms_AbvGrd  1    173.04  926.65
## - Garage_Area   1    173.20  928.55
## - Fireplaces    1    173.73  934.85
## - Bldg_Type     4    175.95  938.01
## - Half_Bath     1    174.18  940.14
## - Lot_Area      1    174.56  944.66
## - Full_Bath     1    185.16 1065.55
## - Overall_Qual  9    220.04 1358.49
##
## Step:  AIC=917.53
## Bonus ~ Lot_Area + Street + Bldg_Type + Overall_Qual + First_Flr_SF +
##      Second_Flr_SF + Full_Bath + Half_Bath + Fireplaces + Garage_Area +
##      TotRms_AbvGrd
##
##              Df Deviance      AIC
```

```

## - Street          1    172.29  910.11
## <none>              172.27  917.53
## - TotRms_AbvGrd   1    173.09  919.69
## - Second_Flr_SF   1    173.18  920.72
## - Garage_Area     1    173.22  921.20
## - Fireplaces      1    173.75  927.37
## - Bldg_Type       4    175.95  930.42
## - Half_Bath       1    174.21  932.82
## - Lot_Area        1    174.57  937.14
## - First_Flr_SF    1    177.40  970.12
## - Full_Bath       1    185.17 1058.01
## - Overall_Qual    9    220.15 1351.85
##
## Step:  AIC=910.11
## Bonus ~ Lot_Area + Bldg_Type + Overall_Qual + First_Flr_SF +
##          Second_Flr_SF + Full_Bath + Half_Bath + Fireplaces + Garage_Area +
##          TotRms_AbvGrd
##
##           Df Deviance    AIC
## <none>           172.29  910.11
## - TotRms_AbvGrd  1    173.11  912.23
## - Second_Flr_SF  1    173.21  913.37
## - Garage_Area    1    173.23  913.67
## - Fireplaces     1    173.77  920.00
## - Bldg_Type      4    176.05  923.90
## - Half_Bath      1    174.24  925.55
## - Lot_Area       1    174.61  929.98
## - First_Flr_SF   1    177.48  963.37
## - Full_Bath      1    185.24 1051.18
## - Overall_Qual   9    220.15 1344.27

```


In the above two approaches we used the BIC selection criteria. Here both forward and backward selection actually picked the same model. Let's check the concordance of this model.

```
#Concordance(train$Bonus, predict(back.model, type = "response"))
```

Not surprisingly, this model outperforms the previous model that we had with a concordance of 96.1%.

Although not covered in detail here, regularized regression can also be applied to logistic regression to get a different view. This might be helpful with the multicollinearity present in these predictor variables. Again, we can use the `glmnet` function with the addition of a `family = "binomial"` option.

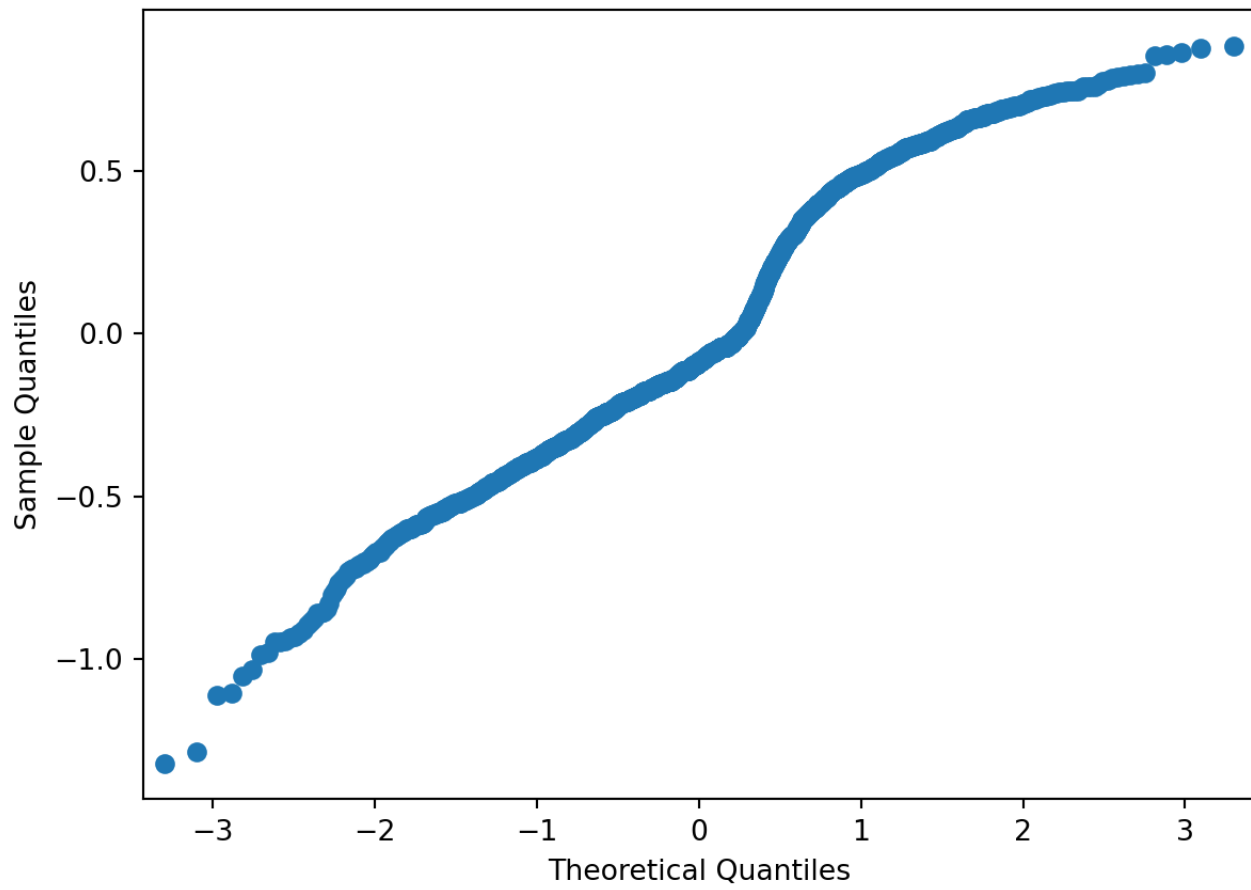
7.4.6 Python Code

Linear Probability Model

```
import statsmodels.formula.api as smf

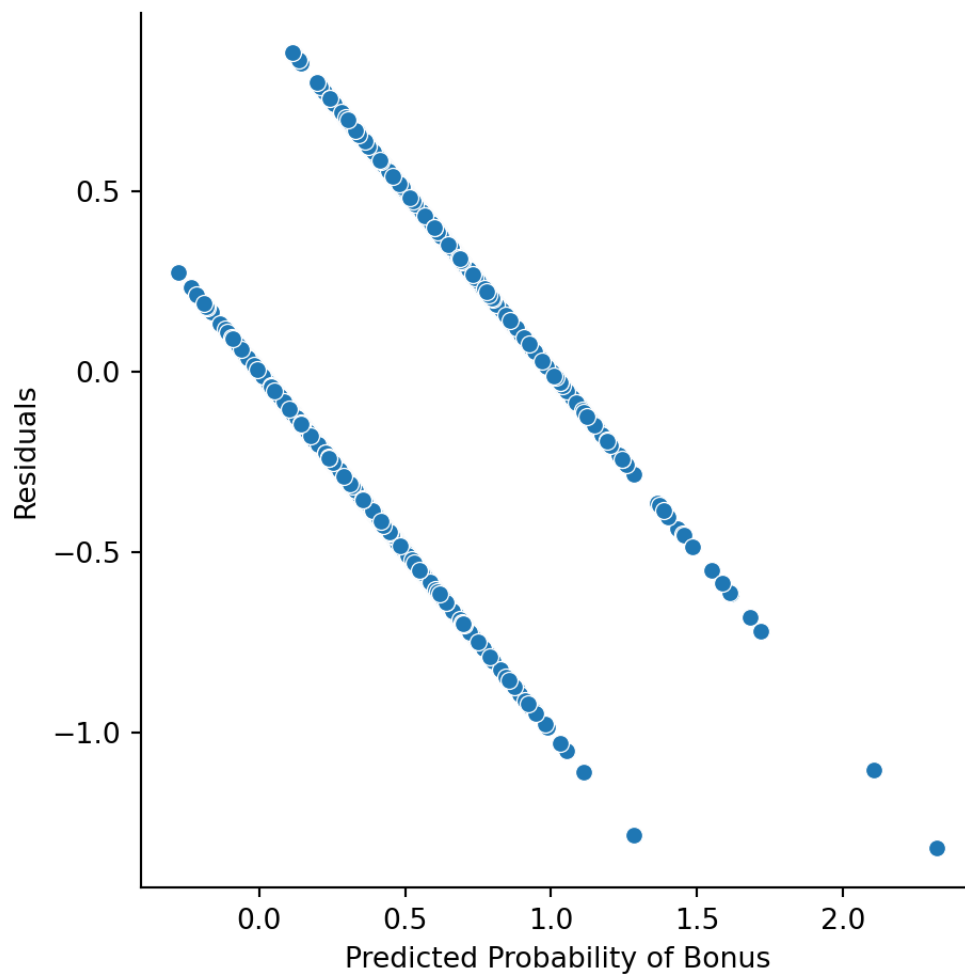
lp_model = smf.ols("Bonus ~ Gr_Liv_Area", data = train).fit()
lp_model.summary()
```

```
sm.api.qqplot(lp_model.resid)  
plt.show()
```



```
train['pred_lp'] = lp_model.predict()  
train['resid_lp'] = lp_model.resid  
  
train[['Bonus', 'pred_lp', 'resid_lp']].head(n = 10)
```

```
plt.cla()
ax = sns.relplot(data = train, y = "resid_lp", x = "pred_lp")
ax.set(ylabel = 'Residuals',
       xlabel = 'Predicted Probability of Bonus')
plt.show()
```



Binary Logistic Regression

```
log_model = smf.logit("Bonus ~ Gr_Liv_Area", data = train).fit()  
log_model.summary()
```

```
odds_ratio = 100*(np.exp(log_model.params) - 1)

print(odds_ratio)
```

Adding Categorical Variables

```
log_model2 = smf.logit("Bonus ~ Gr_Liv_Area + C(Central_Air) + C(Fireplaces)", data)
log_model2.summary()
```



```
odds_ratio = 100*(np.exp(log_model2.params) - 1)

print(odds_ratio)
```

Model Assessment

Python doesn't have concordant / discordant pair calculations. We will learn in Fall semester other metrics to evaluate a logistic regression model that Python does have.

Variable Selection and Regularized Regression

Python does NOT have nice capabilities to do variable selection automatically in statsmodels, scikitlearn, or scipy. All resources I can find involve downloading and installing a package (mlxtend) that is not included by default in anaconda or writing your own function. Scikit learn has something similar but uses the model's coefficients (!!!) to select, not p-values. Scikit learn can do this by evaluating a metric on cross-validation, but that is not covered until machine learning in Fall 3.

